BIRZEIT UNIVERSITY

**Faculty of Engineering & Technology**
**Department of Electrical & Computer Engineering**
**ENCS3390: Operating System Concepts**
**Second Semester, 2025/2026**

# Programming Task 2: Banker's Algorithm

## Objective

Implement the Banker's Algorithm for deadlock avoidance. The system must process resource requests dynamically while ensuring that the system always remains in a safe state.

## Requirements

- Your program must read the initial system configuration from an input file containing:
  - Number of processes (N)
  - Number of resource types (M)
  - MAX matrix
  - AVAILABLE vector

  Example Input Format

  N = 5
  M = 4

  MAX
  7 5 3 4
  3 2 2 2
  9 0 2 2
  2 2 2 2
  4 3 3 3

  AVAILABLE
  3  3 2 2

- Initially, no resources are allocated to any process.

- Design a program that allows the user to interactively enter a request for resources, to release resources, or to output the values of the different data structures (available, maximum, allocation, and need) used in your program.

- Your program must support the following interactive commands:
  - RQ: request resources
  - RL: release resources
  - *:  Display system state
  - RUN: to simulate processes execution
  - EXIT: terminate the program

- To request resources, the user enters:
  - RQ <process_id> r1 r2 ... rm
    **Example:** RQ 0 3 1 2 1
    This means process P0 is requesting resources (3,1,2,1).

- Your program would then output whether the request is granted or denied. If the request is granted, the system should output the safe sequence as well, otherwise, it should output why it is denied.

- To release resources, the user enters:
  - RL <process_id> r1 r2 ... rm
    **Example:** RL 4 1 2 3 1
    This means process P4 releases resources (1,2,3,1).

- Your program must verify that Release <= Allocation. If the release request is invalid, an appropriate error message must be displayed.

- When user enters *, the program must display: Available vector, Max matrix, Allocation matrix, and Need matrix. The output should be formatted clearly in tabular form.

- Your program must support an additional command: RUN
  - If the system is currently in a safe state, the program should:
    - determine a safe sequence,
    - simulate execution of the processes in that sequence,
    - release each process's allocated resources upon completion,
    - display the updated Available vector after each process finishes.
  - If no safe sequence exists, the program should display an appropriate message.

- Your implementation should be modular and well-structured. Suggested functions:
  - load_input()
  - print_state()
  - request_resources()
  - release_resources()
  - is_safe()

- The program must not crash on invalid input and must handle all errors gracefully.

## Remarks:

- **Submission**: Submit your source code and a report (up to 6 pages). Your report must only include test cases demonstrating:
  - A valid request that is granted
  - A request denied due to unsafe state
  - A valid resource release
  - An invalid resource release
  - Multiple sequential requests and releases

- **Demo:** Each group must demonstrate the project to the instructor and answer questions related to the project. Demo dates will be announced later.

- **Groups**: You are allowed to work in groups of **two**.

- **Programming language**: **C or Java or python**.

- **Deadline**: Tuesday 16/6/2026 by midnight.